Vehicle Management System

Project Documentation & Technical Analysis

Java OOP | GUI Application | UML Design

1. Project Overview

The Vehicle Management System is a Java-based desktop application built using Object-Oriented Programming (OOP) principles and a Swing GUI. It allows users to add, display, and manage different types of vehicles (Car, Truck, Motorcycle) stored in a Garage. The system enforces input validation and provides real-time feedback through a graphical interface.

1.1 System Goals

- Provide a structured, extensible model for managing vehicles of different types.
- Enforce robust input validation with informative error messages.
- Display all registered vehicles with full details in a user-friendly interface.
- Demonstrate OOP concepts: inheritance, abstraction, encapsulation, and polymorphism.

1.2 Technology Stack

Component	Technology
Programming Language	Java (SE)
UI Framework	Java Swing (JFrame, JPanel, JTextArea, JOptionPane)
Architecture	OOP with Inheritance & Abstraction
Design Notation	UML Class Diagram
IDE	Any Java IDE (IntelliJ, Eclipse, NetBeans)

2. Requirements Analysis

2.1 Functional Requirements

ID	Requirement	Description
NR1	Create Vehicle	User can create a Car, Truck, or Motorcycle with specific attributes.
NR2	Add to Garage	The created vehicle is added to the Garage (max 100 vehicles).
NR3	Display Vehicles	All vehicles are shown with full details in the text output area.
NR4	Input Validation	Fields are validated for empty values, numeric types, and positive ranges.
NR5	Graphical UI	A clean, user-friendly GUI with dynamic fields based on vehicle type.

2.2 Non-Functional Requirements

ID	Requirement	Description
NFR1	Performance	Vehicle addition and display should complete in under 1 second.
NFR2	Usability	Error messages must be clear, readable, and actionable.
NFR3	Extensibility	Code structure must support adding new vehicle types with minimal changes.

2.3 Use Cases

Actor: User

Use Case	Precondition	Postcondition
Add Vehicle	Form fields are filled correctly.	Vehicle added to Garage; output area updated.
Display Vehicles	At least one vehicle exists in Garage.	All vehicles displayed in output area.
Handle Validation Error	User submits form with invalid data.	JOptionPane shows descriptive error message; form not cleared.

3. System Architecture & Class Design

3.1 Architecture Overview

The system is structured around a layered OOP model. The abstract Vehicle class forms the core of the domain model, with three concrete subclasses. The Garage acts as a repository, and the VehicleManagementGUI serves as the presentation and controller layer.

3.2 Class Descriptions

3.2.1 Vehicle (Abstract Class)

The root class of the vehicle hierarchy. Defines shared attributes and declares the abstract method displayInfo() which forces all subclasses to implement their own display logic.

Member	Type	Description
ownerName	String	Full name of the vehicle owner.
brand	String	Vehicle brand/manufacturer (e.g., Toyota).
model	String	Specific model name (e.g., Corolla).
year	int	Manufacturing year (validated numerically).
displayInfo()	abstract void	Must be overridden by each subclass to print details.

3.2.2 Car (extends Vehicle)

Represents a passenger vehicle. Adds the seating capacity attribute specific to cars.

Member	Type	Description
seatingCapacity	int	Number of passenger seats (must be positive integer).
getSeatingCapacity()	int	Returns the seating capacity.
setSeatingCapacity(int)	void	Sets the seating capacity.
displayInfo()	void	Overrides Vehicle; prints all car details.

3.2.3 Truck (extends Vehicle)

Represents a cargo vehicle. Adds the cargo capacity attribute expressed as a decimal (double).

Member	Type	Description
cargoCapacity	double	Cargo weight capacity in tons (must be positive).
getCargoCapacity()	double	Returns cargo capacity.
setCargoCapacity(double)	void	Sets cargo capacity.
displayInfo()	void	Overrides Vehicle; prints all truck details.

3.2.4 Motorcycle (extends Vehicle)

Represents a two-wheeled vehicle. Adds engine size (in cc) as the distinguishing attribute.

Member	Type	Description
engineSize	int	Engine displacement in cubic centimeters (cc).
getEngineSize()	int	Returns engine size.
setEngineSize(int)	void	Sets engine size.
displayInfo()	void	Overrides Vehicle; prints all motorcycle details.

3.2.5 Garage

Acts as the central repository for all vehicles. Manages a fixed-size array with a counter, enforces the maximum capacity, and provides operations for CRUD management of vehicles.

Member	Return Type	Description
vehicles: Vehicle[]	—	Array storing up to 100 vehicles.
count: int	—	Current number of vehicles stored.
MAX_VEHICLES: int	—	Constant maximum size (100).
addVehicle(Vehicle)	boolean	Adds vehicle if capacity not exceeded; returns success.
removeVehicle(String ownerName)	boolean	Removes vehicle by owner name; shifts array.
updateVehicle(...)	boolean	Finds vehicle by owner and updates all fields.
displayVehicles()	void	Prints summary of all vehicles.
displayVehicleDetails(String)	void	Prints full details for a specific vehicle.
getAllVehicleInfo()	String	Returns formatted string of all vehicle details for GUI.
getCount()	int	Returns the current vehicle count.

3.2.6 VehicleManagementGUI

The main application window. Extends JFrame and wires together all the UI components with the domain logic. Listens for button events and delegates business logic to the Garage.

Member	Type	Description
garage	Garage	The single Garage instance used by the GUI.
ownerName, brand, model	JTextField	Input fields for vehicle base info.
year	JTextField	Year field; validated as numeric and in range.
vehicleType	JComboBox	Dropdown: Car / Truck / Motorcycle.

Member	Type	Description
dynamicField	JTextField	Changes label/placeholder based on selected type.
outputArea	JTextArea	Scrollable display for all vehicle information.
updateDynamicField()	void	Updates dynamic field label based on combo selection.
AddVehicleListener	ActionListener	Validates input, creates vehicle, adds to Garage.
main(String[])	static void	Application entry point.

4. Class Relationships

4.1 Inheritance (Generalization)

The three concrete classes extend the abstract Vehicle class. This implements the Is-A relationship and enables polymorphism across the system.

Subclass	Superclass	Relationship Type	Multiplicity
Car	Vehicle	Generalization (extends)	1 Car : 1 Vehicle
Truck	Vehicle	Generalization (extends)	1 Truck : 1 Vehicle
Motorcycle	Vehicle	Generalization (extends)	1 Motorcycle : 1 Vehicle

4.2 Composition / Association

Source	Target	Relationship Type	Multiplicity
Garage	Vehicle	Composition (has-a)	1 Garage : 0..100 Vehicles
VehicleManagementGUI	Garage	Association (uses-a)	1 GUI : 1 Garage

4.3 Relationship Directions

- Garage knows about Vehicle instances (uni-directional). Vehicle does not know about Garage.
- VehicleManagementGUI calls methods on Garage (uni-directional). Garage does not reference the GUI.
- Inheritance arrows point from subclass to superclass (hollow triangle arrowhead in UML).

5. Sequence Diagram — Add Vehicle Flow

The following sequence describes the interaction between components when the user submits the Add Vehicle form:

1. User fills in all form fields (ownerName, brand, model, year, type, dynamic field).
2. User clicks the 'Add Vehicle' button.
3. AddVehicleListener.actionPerformed() is triggered in VehicleManagementGUI.
4. GUI reads and validates all field values (empty check, year numeric check, extra field positive check).
5. If validation fails: JOptionPane.showMessageDialog() displays the error; flow stops.
6. If validation passes: GUI instantiates the correct subclass (Car / Truck / Motorcycle).
7. GUI calls garage.addVehicle(vehicleInstance).
8. Garage checks if count < MAX_VEHICLES; stores the vehicle in vehicles[] and increments count.
9. Garage returns true to GUI.
10. GUI calls garage.getAllVehicleInfo() and displays result in outputArea.
11. GUI clears all input fields.

5.1 Exception Flow

- Empty field detected: JOptionPane error — 'Please fill in all fields.'
- Year not numeric: JOptionPane error — 'Year must be a valid number.'
- Year out of range (e.g., < 1886 or > current year): JOptionPane error.
- Dynamic field value negative or zero: JOptionPane error — 'Value must be a positive number.'
- Garage full (100 vehicles): addVehicle() returns false; GUI shows error.

6. OOP Concepts Demonstrated

OOP Concept	Where Applied	How
Abstraction	Vehicle class	Declares abstract displayInfo(); hides implementation details from Garage.
Inheritance	Car, Truck, Motorcycle	Each extends Vehicle and inherits base attributes and getters/setters.
Polymorphism	Garage.vehicles[]	Stores Vehicle references; calls displayInfo() polymorphically on each element.
Encapsulation	All classes	All attributes are private; accessed only via public getters and setters.
Composition	Garage	Owns and manages Vehicle[] — vehicles only exist within the Garage context.

7. Input Validation Logic

7.1 Validation Rules

Field	Rule	Error Message
ownerName	Not empty	"Owner name cannot be empty."
brand	Not empty	"Brand cannot be empty."
model	Not empty	"Model cannot be empty."
year	Numeric, 1886 <= year <= current year	"Please enter a valid year."
seatingCapacity (Car)	Positive integer	"Seating capacity must be a positive number."
cargoCapacity (Truck)	Positive double	"Cargo capacity must be a positive number."
engineSize (Motorcycle)	Positive integer	"Engine size must be a positive number."

7.2 Dynamic Field Behavior

The dynamic input field changes its label and expected input type based on the selected vehicle type in the JComboBox:

- Car selected → Label: 'Seating Capacity' | Input: positive int
- Truck selected → Label: 'Cargo Capacity (tons)' | Input: positive double
- Motorcycle selected → Label: 'Engine Size (cc)' | Input: positive int

8. Key Code Snippets

8.1 Vehicle Abstract Class

```
public abstract class Vehicle {
    private String ownerName, brand, model;
    private int year;

    public Vehicle(String ownerName, String brand, String model, int year) {
        this.ownerName = ownerName;
        this.brand = brand;
        this.model = model;
        this.year = year;
    }

    public abstract void displayInfo();
    // getters and setters...
}
```

8.2 Garage — addVehicle()

```
public boolean addVehicle(Vehicle v) {
    if (count >= MAX_VEHICLES) return false;
    vehicles[count++] = v;
    return true;
}
```

8.3 GUI — AddVehicleListener (Simplified)

```
String type = (String) vehicleTypeCombo.getSelectedItem();
Vehicle v;
if (type.equals("Car")) {
    int seats = Integer.parseInt(dynamicField.getText().trim());
    v = new Car(owner, brand, model, year, seats);
} else if (type.equals("Truck")) {
    double cargo = Double.parseDouble(dynamicField.getText().trim());
    v = new Truck(owner, brand, model, year, cargo);
} else {
    int engine = Integer.parseInt(dynamicField.getText().trim());
    v = new Motorcycle(owner, brand, model, year, engine);
}
garage.addVehicle(v);
outputArea.setText(garage.getAllVehicleInfo());
```

9. Project Summary

Aspect	Summary
Language	Java SE
UI	Java Swing (JFrame-based GUI)
Design Pattern	Layered OOP (Domain + Repository + Presentation)
Core Classes	Vehicle (abstract), Car, Truck, Motorcycle, Garage, VehicleManagementGUI
Key OOP Features	Abstraction, Inheritance, Polymorphism, Encapsulation
Garage Capacity	Maximum 100 vehicles (array-based)
Validation	All fields validated; errors shown via JOptionPane
Extensibility	New vehicle types require only: extends Vehicle + override displayInfo()

End of Document